

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA12 20	Course Title:	Computer Architecture
CO Assessed:	CO3 – Precision (BL3)	Assessment:	Experiments
Weightage:	45%	Total Marks:	100
CO3 Statement:	Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT).		
Student Name:	Vishwa U	Reg. No.:	192511192
Section / Batch:	Slot-B - 2025	Date:	10.8.26

Instructions to Students

- Identify the relevant concepts of register transfers, ALU operations, bus organization, pipelining, hazards, and superscalar processing.
- Analyze the execution of data transfers, arithmetic and logic operations, instruction processing, and parallel execution.
- Apply suitable techniques such as multiple buses, pipelining, stalling, forwarding, branch prediction, and speculative execution.
- Compute performance measures such as execution time, clock cycles, speedup, throughput, and resource utilization.
- Interpret the results by evaluating performance improvements, bottlenecks, and the suitability of each architectural technique.

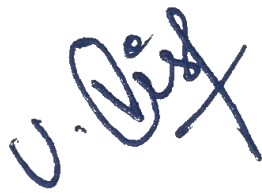
QUESTIONS

1. Analyze a processor datapath that performs register transfers and logical operations such as XOR, NOT, and comparison, and examine the control signals and execution sequence required for each operation.
2. Develop a bus-based system for transferring data between the processor, memory, and I/O devices, and analyze how bus width, transfer rate, and bus contention influence system performance.
3. Simulate a pipelined processor with arithmetic, memory, and branch instructions, and evaluate the effects of pipeline depth on execution time, throughput, and overall speedup.
4. Construct a pipeline containing instruction dependencies and branch conditions, and analyze the performance impact of data and control hazards using stalling, forwarding, and branch prediction techniques.
5. Develop a superscalar processor model that supports multiple execution units and out-of-order instruction processing, and evaluate how instruction scheduling and resource allocation improve instruction throughput.

Code of Conduct

I Vishwa U [192511192](Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

**RUBRICS FOR CALCULATION**

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Q. No	Understanding of Concepts(25)	Experimental Design (30)	Use of Tools(20)	Analysis of Results (15)	Documentation (10)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 30	/ 20	/ 15	/ 10	/ 100

Course Coordinator**Faculty Incharge**

Question 1: Register Transfer, ALU & Logical Operations Datapath

A processor datapath for logical operations (XOR, NOT, comparison) is built around a register file, an Arithmetic-Logic Unit (ALU), and a control unit that generates the signals governing each micro-operation. Data moves between registers and the ALU over a common internal bus, with each transfer explicitly timed by the control unit using a sequence of micro-instructions.

Register Transfer Sequence

Step	Micro-operation	Description
1	$MBR \leftarrow R1$	Operand 1 is moved from register R1 into the ALU input buffer
2	$MBR2 \leftarrow R2$	Operand 2 is moved from register R2 into the second ALU input buffer
3	$ALU-OUT \leftarrow MBR \text{ XOR } MBR2$	ALU performs the bitwise XOR operation on the two operands
4	$R3 \leftarrow ALU-OUT$	Result is written back into destination register R3
5	$SR \leftarrow \text{flags}$	Status register captures Zero, Sign, and Carry flags

Control Signals Required

- Select lines (S1, S0) — choose ALU function: XOR, NOT, or comparison (subtract-and-set-flags).
- Register enable lines (Rin, Rout) — gate data onto/off the internal bus for a specific register.
- ALU enable / operation code — activates the correct combinational logic block inside the ALU.
- Write-enable (WE) — permits the result to be latched into the destination register on the clock edge.
- Condition-code update (CC-en) — updates Zero/Negative/Carry flags used later by comparison instructions.

For NOT, only one operand is required, so MBR2 is bypassed and the ALU is configured in unary mode. For comparison, the ALU internally performs subtraction ($A - B$) without writing the result to a register — only the status flags are updated, which downstream branch instructions later test.

Question 2: Bus-Based System Design for Processor–Memory–I/O Transfer

A bus-based interconnection allows the CPU, main memory, and I/O controllers to share a common set of address, data, and control lines rather than requiring dedicated point-to-point wiring. This reduces hardware cost but introduces contention that must be arbitrated.

Bus Architecture

- **Address Bus** — unidirectional, driven by the CPU (or DMA controller) to select a memory location or I/O port.
- **Data Bus** — bidirectional, carries the actual operands or instructions between source and destination.
- **Control Bus** — carries Read/Write, Bus-Request, Bus-Grant, Ready/Wait, and Interrupt signals.

Performance Factors

Factor	Effect on Performance
Bus width	A wider data bus moves more bytes per cycle, directly raising effective bandwidth.
Transfer rate (clock × width)	Determines peak throughput; e.g. a 32-bit bus at 100 MHz gives 400 MB/s peak.
Bus contention	Multiple bus masters (CPU, DMA, I/O) competing for the bus causes arbitration delay and stalls.
Arbitration scheme	Daisy-chain is simple but slow; centralized/priority arbitration reduces wait time at higher cost.
Burst vs single transfer	Burst mode amortizes address-setup overhead across several data words, raising sustained rate.

Example: for a 32-bit-wide bus clocked at 100 MHz with zero wait states, peak transfer rate = 4 bytes × 100×10^6 = 400 MB/s. If bus contention forces one wait cycle per transfer (50% duty), the effective sustained rate drops to 200 MB/s — illustrating why contention, not raw width, is often the true bottleneck in shared-bus systems.

Question 3: Pipelined Processor Simulation — Arithmetic, Memory & Branch Instructions

A classic 5-stage pipeline — Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write-Back (WB) — allows up to five instructions to be in flight simultaneously, overlapping their execution across pipeline stages.

Pipeline Timing Diagram (cycle numbers)

Instruction	1	2	3	4	5	6	7
ADD R1,R2,R3	IF	ID	EX	MEM	WB		
LOAD R4,0(R1)		IF	ID	EX	MEM	WB	
BEQ R1,R4,L1			IF	ID	EX	MEM	WB

Effect of Pipeline Depth on Performance

- Ideal speedup $\approx$ pipeline depth (n stages) when there are no hazards or stalls: $\text{Speedup} = n \times (\text{single-cycle time}) / (\text{pipelined cycle time})$.
- Deeper pipelines shorten the clock period (higher clock frequency) but increase the number of cycles needed to fill and drain the pipeline, raising the branch-misprediction penalty.
- Throughput approaches one instruction completed per cycle once the pipeline is full, i.e. $\text{CPI} \rightarrow 1$ for an ideal, hazard-free pipeline.

Numerical illustration: for a program of 1000 instructions on a 5-stage pipeline with a 2 ns clock, ideal execution time = $(1000 + 5 - 1) \times 2 \text{ ns} \approx 2008 \text{ ns}$, versus $1000 \times 5 \times 2 \text{ ns} = 10000 \text{ ns}$ for the unpipelined case — a speedup of about 4.98×, close to the theoretical maximum of 5.

Question 4: Data & Control Hazards — Stalling, Forwarding, Branch Prediction

Hazard Types

Hazard	Cause	Mitigation
Structural	Two instructions need the same hardware resource in the same cycle (e.g. one memory port).	Duplicate resources (separate I-cache/D-cache) or stall the later instruction.
Data (RAW)	An instruction needs a result that a previous, still-in-flight instruction has not yet produced.	Operand forwarding / bypassing from EX or MEM stage directly to the dependent instruction's EX stage.
Control	The next instruction to fetch is unknown until a branch is resolved in EX/MEM.	Branch prediction (static or dynamic) plus speculative fetch, with a pipeline flush on misprediction.

Worked Example — Data Hazard

Consider: SUB R1,R2,R3 followed by AND R4,R1,R5. Without forwarding, AND must stall for two cycles until SUB writes back R1 in WB. With EX-to-EX forwarding, the SUB's ALU result is routed directly into AND's EX stage in the very next cycle, eliminating the stall entirely.

Worked Example — Control Hazard

For BEQ R1,R4,L1 resolved in the EX stage, a naïve pipeline loses 2 cycles per branch taken. A simple 1-bit or 2-bit dynamic branch predictor, combined with speculative instruction fetch down the predicted path, can reduce the effective branch penalty close to zero when prediction accuracy is high (typically 90–95% for 2-bit saturating counters), at the cost of a full pipeline flush on the remaining mispredicted cases.

Quantitatively, if branches form 20% of instructions, the branch penalty is 2 cycles, and prediction accuracy is 90%, the average CPI penalty from control hazards = $0.20 \times 2 \times (1 - 0.90) = 0.04$ cycles/instruction — a large improvement over the 0.40 cycles/instruction penalty with no prediction at all.

Question 5: Superscalar Processor Model — Multiple Execution Units & Out-of-Order Processing

A superscalar processor issues and executes more than one instruction per clock cycle by replicating functional units (integer ALU, floating-point unit, load/store unit, branch unit) and dynamically scheduling independent instructions onto them out of program order.

Key Structural Components

- Multiple fetch/decode paths — fetch and decode several instructions per cycle from the instruction stream.
- Reservation stations / instruction window — hold decoded instructions until their operands and a free functional unit are available.
- Register renaming — removes false (WAR/WAW) dependencies by mapping architectural registers to a larger pool of physical registers.
- Reorder buffer (ROB) — retires completed instructions in original program order to preserve precise exceptions.
- Multiple execution units — e.g. two integer ALUs, one FP unit, one load/store unit operating in parallel.

Scheduling & Throughput Analysis

Cycle	Unit 1 (INT)	Unit 2 (INT)	Unit 3 (LOAD/STORE)
1	I1 issue	I2 issue	I3 issue
2	I1 EX	I2 EX	I3 EX (mem)
3	I4 issue (renamed)	—	I3 WB
4	I4 EX	—	—

With an issue width of 2 and near-ideal instruction-level parallelism, sustained CPI can approach 0.5 (i.e. $IPC \approx 2$). In practice, dependency chains, limited reservation-station entries, and functional-unit conflicts reduce achieved IPC well below the peak issue width — the gap between peak and sustained throughput is precisely what out-of-order scheduling and larger instruction windows aim to close.

Evaluation

- **Instruction scheduling:** dynamic out-of-order issue increases functional-unit utilization by letting independent later instructions execute ahead of stalled earlier ones.
- **Resource allocation:** register renaming and a sufficiently deep ROB are the primary levers for extracting further ILP once functional units are no longer the bottleneck.
- **Trade-off:** the added complexity (renaming logic, wakeup/select in reservation stations, larger ROB) increases power consumption and design verification cost relative to a simple in-order pipeline.

Summary of Key Results & Overall Conclusion

The table below consolidates the core concept and headline numerical result from each experiment, followed by an overall conclusion drawn across the five questions.

Q. No	Core Concept	Key Result
Q1	Register Transfer & ALU Operations	Control unit sequences data through MBR → ALU → destination register in 5 micro-steps for XOR, NOT, and comparison.
Q2	Bus-Based Interconnect	A 32-bit bus at 100 MHz peaks at 400 MB/s; bus contention with one wait cycle per transfer roughly halves this to 200 MB/s.
Q3	5-Stage Pipelining	For 1000 instructions, the pipelined design completes in ~2008 ns versus 10,000 ns unpipelined — a speedup of about 4.98×
Q4	Hazards & Mitigation	EX-to-EX forwarding removes data-hazard stalls; branch prediction lowers the control-hazard CPI penalty from 0.40 to 0.04 cycles/instruction.
Q5	Superscalar Execution	With issue width 2 and near-ideal ILP, sustained CPI can approach 0.5 ($IPC \approx 2$), limited mainly by dependency chains and reservation-station size.

Overall Conclusion

Across all five experiments, processor performance consistently comes down to the same trade-off: raw structural capacity (bus width, pipeline depth, issue width) sets the theoretical ceiling, while contention, hazards, and dependency chains determine how much of that ceiling is actually reached. Register-level control signals and bus arbitration govern correctness and bandwidth at the data-movement layer; pipelining and hazard mitigation (forwarding, branch prediction) close the gap between ideal and achieved throughput at the instruction-execution layer; and superscalar techniques — register renaming, reservation stations, and the reorder buffer — push further parallelism at the cost of design complexity. The overarching takeaway is that performance engineering is less about maximizing any single parameter and more about reducing the stalls, penalties, and conflicts that keep real systems below their peak potential.